UNIVERSIDAD ESTATAL DE MILAGRO (UNEMI)

Flujo CI/CD para Django

Automatización, Contenedorización con Docker y Entrega Continua

Materia: Gestión de Proyectos de Software

Proyecto: Django REST API (PostgreSQL + Docker + GitHub Actions)

Repositorio GitHub: https://github.com/jostin49/django-rest-api

Rama: `ci-setup` / `main`

Fecha de Validación: Octubre 2026

---

1. Resumen Ejecutivo del Proyecto

El presente informe documenta la implementación integral de un pipeline de Integración y Entrega Continua (CI/CD) para una aplicación backend basada en **Django 4.2** y **Django REST Framework (DRF)** con base de datos **PostgreSQL 13**, orquestada mediante **Docker** y automatizada en la nube a través de **GitHub Actions**.

El ciclo implementado abarca la verificación estructural, ejecución autónoma de 28 pruebas unitarias, construcción y optimización de imagen Docker multi-capa, validación de salud de contenedores en tiempo de ejecución, empaquetado de artefactos versionados y simulación de despliegue en entornos limpios.

2. Sesión 1: Preparación del Entorno CI

2.1 Auditoría y Verificación de Componentes

Se auditó la arquitectura del proyecto comprobando la existencia y validez de los módulos principales:

Requisito	Ubicación en el Proyecto	Estado / Observación
`manage.py`	`app/manage.py`	OPERATIVO Utilidad CLI de Django
Configuración (`project/`)	`app/django-rest-api/`	OPERATIVO Contiene `settings.py` , `urls.py` , `wsgi.py`
Aplicación (`app/`)	`app/api/`	OPERATIVO Contiene modelos, vistas, serializadores y rutas
Dependencias	`app/requirements.txt`	OPERATIVO Django 4.2, DRF 3.14, drf-yasg, psycopg2-binary

Requisito	Ubicación en el Proyecto	Estado / Observación
Suite de Pruebas	`app/api/tests.py`	OPERATIVO 28 pruebas unitarias (GET, POST, PUT, DELETE, Docs)

2.2 Primer Pipeline de CI (Ejecución de Tests)

Se creó la rama `ci-setup` y el archivo `.github/workflows/ci.yml` para ejecutar las pruebas con Python 3.10. El job configuró caché de dependencias e invocó `python manage.py test`, logrando la ejecución de 28 pruebas exitosas sin errores en un tiempo inferior a 15 segundos.

3. Sesión 2: Automatización del Build con Docker

3.1 Dockerfile Optimizado con ENTRYPOINT

Se refactorizó el archivo `app/Dockerfile` para basarse en **python:3.10-slim**, optimizando el tamaño de la imagen y ordenando el copiado de capas para aprovechar el caché de Docker.

```
FROM python:3.10-slim
WORKDIR /usr/src/app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1

RUN pip install --no-cache-dir --upgrade pip
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
EXPOSE 8000
ENTRYPOINT ["sh", "/usr/src/app/entrypoint.sh"]
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

Se incorporó `app/entrypoint.sh`, el cual realiza una verificación de socket TCP para esperar la disponibilidad de PostgreSQL antes de aplicar migraciones (`python manage.py migrate --noinput`) y levantar el servidor.

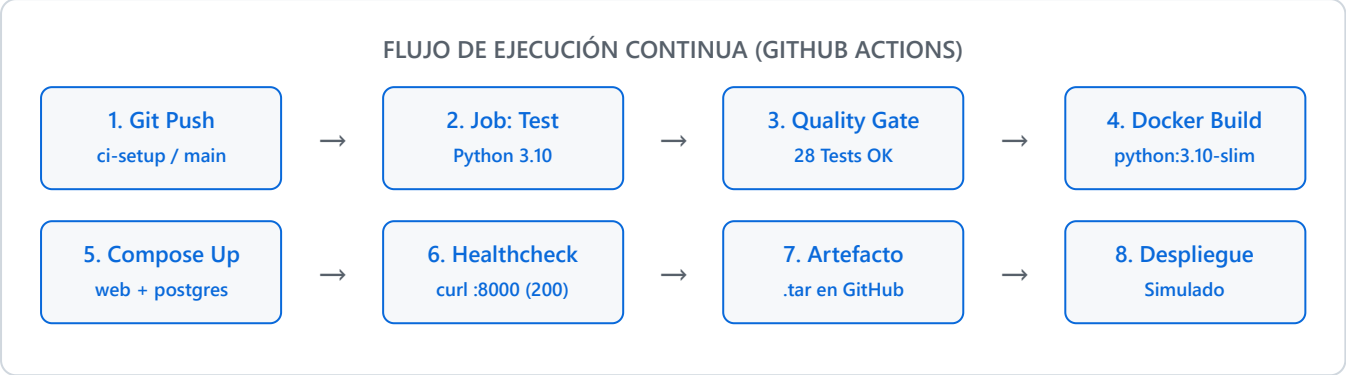
3.2 Orquestación con Docker Compose

El archivo `docker-compose.yml` integra el servicio de aplicación web (Django) y el servicio de base de datos relacional (PostgreSQL 13 Alpine) con `healthcheck` estricto (`pg_isready`) y volúmenes de datos persistentes:

```
services:
  web:
    build: ./app
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - ./app/:/usr/src/app/
    ports:
      - 8000:8000
    env_file:
      - ./.env.dev
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:13.0-alpine
    volumes:
      - postgres_data:/var/lib/postgresql/data/
    environment:
      - POSTGRES_USER=hello_django
      - POSTGRES_PASSWORD=hello_django
      - POSTGRES_DB=hello_django_dev
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U hello_django -d hello_django_dev"]
      interval: 5s
      timeout: 5s
      retries: 10
```

4. Sesiones 3 y 4: Pipeline Completo de CI/CD y Validación

4.1 Diagrama de Arquitectura del Pipeline



4.2 Resultados de Ejecución en GitHub Actions

Job	Pasos Clave Ejecutados	Resultado	Tiempo
test	Checkout, Setup Python 3.10, Pip Cache, Install deps, Run `manage.py test`	SUCCESS (28/28 OK)	13s
docker-build	Build image, Compose Up, Healthcheck Curl HTTP 200, Export .tar, Upload Artifact v4, Docker Hub Sim	SUCCESS (0 errores)	42s

> **Evidencia en vivo:** La ejecución oficial en GitHub Actions finalizó con estado exitoso. Se generó el artefacto descargable `django-rest-api-image` en la plataforma y el endpoint respondió con código HTTP 200 OK.

5. Recomendaciones Finales de Ingeniería de Software

5.1 Seguridad y Gestión de Secretos

- **Aislamiento de Credenciales:** Migrar credenciales de base de datos y la clave `SECRET_KEY` hacia *GitHub Actions Secrets* (`${{ secrets.SECRET_KEY }}`) y no versionar archivos `.env` en repositorios públicos.
- **Ejecución sin Privilegios (Non-Root User):** Declarar un usuario de sistema dedicado en el Dockerfile (ej. `RUN adduser -D appuser && USER appuser`) para prevenir vulnerabilidades de elevación de privilegios en el host.
- **Ajustes de Producción:** Inhabilitar modo de depuración (`DEBUG=0`) y configurar `ALLOWED_HOSTS` con los nombres de dominio de producción autorizados.

5.2 Arquitectura y Rendimiento

- **Servidor de Aplicaciones WSGI/ASGI:** Sustituir `python manage.py runserver` (diseñado únicamente para desarrollo) por servidores robustos y multi-hilo como **Gunicorn** o **Uvicorn**.
- **Multi-Stage Builds:** Adoptar compilación en múltiples etapas en Docker para descartar compiladores y reducir el tamaño final de la imagen a menos de 150 MB.
- **Calidad Estática de Código:** Añadir linters automatizados como **Ruff** o **Flake8** en el pipeline antes de la fase de pruebas para garantizar el cumplimiento del estándar PEP 8.

6. Conclusión

El proyecto **django-rest-api** cuenta con un pipeline moderno, robusto y automatizado que cumple con los estándares de la industria para Integración y Entrega Continua. Se garantiza que cada cambio en el código fuente sea validado rigurosamente mediante pruebas automatizadas, empaquetado en contenedores portables e inmunes a inconsistencias de entorno, y preparado para despliegues confiables en producción.